

1 WHAT MONGODB IS GOOD AT

- ✓ Document database with flexible schema.
- ✓ Best when data is read together and can be stored together.
- ✓ Great for catalogs, profiles, events, content, carts, and rapidly evolving schemas.
- ✓ Think in access patterns first, tables second.

2 WHEN TO USE / WHEN NOT TO USE



USE WHEN

- JSON-like documents fit the domain.
- High read/write scale and evolving schema.
- Aggregation pipelines and denormalized reads help.



AVOID OR REVIEW CAREFULLY WHEN

- Complex cross-document joins dominate.
- Strict relational reporting is central.
- Unbounded arrays or oversized documents are likely.

3 SPRING BOOT SETUP

Maven dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Optional -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

application.yml

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://user:***@
        cluster0.example.mongodb.net/
        shop?retryWrites=true&w=majority
      database: shop
      auto-index-creation: false
```

4 MINIMAL IMPLEMENTATION FLOW



Entity example

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document("products")
public class Product {
    @Id
    private String id;
    private String name;
    private BigDecimal price;
    // getters and setters
}
```

Repository example

```
import org.springframework.data.mongodb.repository.MongoRepository;

public interface ProductRepository
    extends MongoRepository<Product, String> {
    // custom queries if needed
}
```

5 ☆ GOLDEN RULES



Model for the query.



Embed first, reference when needed.



Add indexes deliberately.



Measure before tuning.



Keep examples production-minded.

1 KEY ANNOTATIONS

	@Document	Marks a class as a MongoDB document and sets collection name.	<code>@Document("orders")</code> <code>class Order { }</code>
	@Id	Marks the primary key field.	<code>@Id</code> <code>private String id;</code>
	@Indexed	Creates an index on a field.	<code>@Indexed</code> <code>private String customerId;</code>
	@CompoundIndex	Creates a compound index on multiple fields.	<code>@CompoundIndex(def = "{ 'customerId': 1, 'status': 1 }", name = "idx_customer_status")</code>
	@Version	Enables optimistic locking for the document.	<code>@Version</code> <code>private Long version;</code>
	@CreatedDate	Auto-populates field on document creation.	<code>@CreatedDate</code> <code>private Instant createdAt;</code>
	@LastModifiedDate	Auto-updates field on every modification.	<code>@LastModifiedDate</code> <code>private Instant updatedAt;</code>
	@DocumentReference (lazy = true)	Lightweight reference to another document by id (no DBRef overhead).	<code>@DocumentReference(lazy = true)</code> <code>private Customer customer;</code>
	DBRef (exists)	Built-in MongoDB reference type. Use carefully (extra database round trips).	<code>@DBRef</code> <code>private Customer customer;</code>

2 ENTITY EXAMPLE

```
@Document("orders")
@CompoundIndex(def = "{ 'customerId': 1, 'status': 1 }", name = "idx_customer_status")
public class Order {
    @Id
    private String id;

    @Indexed
    private String customerId;
    private OrderStatus status;
    private BigDecimal totalAmount;
    private List<OrderItem> items;

    @Version
    private Long version;

    @CreatedDate
    private Instant createdAt;

    @LastModifiedDate
    private Instant updatedAt;
}
```

3 REPOSITORY VS MONGOTEMPLATE

MONGOREPOSITORY (SPRING DATA)

- Quick CRUD operations
- Derived query methods
- Paging & sorting support
- Less boilerplate

```
public interface OrderRepository extends MongoRepository<Order, String> {
    List<Order> findByCustomerIdAndStatus(String customerId, OrderStatus status);
    Page<Order> findByStatus(String status, Pageable pageable);
}
```

MONGOTEMPLATE (FINE-GRAIN CONTROL)

- Dynamic queries and criteria
- Updates and bulk operations
- Aggregation pipelines
- Finer control over operations

```
Query query = Query.query(Criteria.where("status").is("PAID"));
List<Order> orders = mongoTemplate.find(query, Order.class);
```

4 SERVICE FLOW



```
@Service
public class OrderService {
    private final OrderRepository orderRepository;

    public Order save(Order order) {
        validate(order);
        order.setStatus(order.getStatus() == null ? OrderStatus.NEW : order.getStatus());
        return orderRepository.save(order);
    }
}
```

5 AUDITING + CONCURRENCY

-  Enable `@EnableMongoAuditing` in a `@Configuration` class.
-  `@Version` helps with optimistic locking.
-  Keep document updates focused and intentional.
-  Use update operators (where possible) to avoid full document rewrites.

6 IMPLEMENTATION TIPS

-  Prefer projection for read APIs.
-  Avoid giant documents.
-  Define indexes intentionally.
-  Keep collections aligned with access patterns.
-  Test with realistic data volumes.



MODELING FIRST PRINCIPLES

- ✓ Model for query patterns.
- ✓ Embed when data is read together.
- ✓ Reference when cardinality is high or duplication is hard.
- ✓ Keep documents bounded.
- ✓ Design indexes with schema.

1 EMBEDDED DOCUMENT PATTERN

- ✓ **Use when** Child data is read with parent and updated together.
- **Example** Order with `orderItems` and `shippingAddress` embedded.
- ⚠ **Watch out** Document growth.

2 REFERENCE / EXTENDED REFERENCE

- ✓ **Use when** Related data changes independently or cardinality is high.
- **Example** Order stores `customerId` and a few duplicated customer fields like `customerName`.
- ⚠ **Watch out** Extra lookups and stale duplicated fields.

3 ATTRIBUTE PATTERN

- ✓ **Use when** For variable attributes and searchable product specs.
- **Example** `product.attributes = [{k:"color",v:"black"}, {k:"size",v:"XL"}]`.
- ⚠ **Watch out** Index only fields you query.

4 BUCKET PATTERN

- ✓ **Use when** For time-series or event streams grouped by time/user/device.
- **Example** Page views or sensor readings grouped per hour/day.
- ⚠ **Watch out** Bucket size and write amplification.

5 SUBSET + OUTLIER PATTERN

- ✓ **Use when** To keep hot data small and move large or rare fields elsewhere.
- **Example** Product summary in main document, long description or huge review list in another collection.
- ⚠ **Watch out** Decide hot vs cold fields deliberately.

6 COMPUTED / POLYMORPHIC PATTERN

- | COMPUTED VALUES | | POLYMORPHIC COLLECTION | |
|--------------------|--|------------------------|---|
| ✓ Use when | Computed values like <code>orderTotal/reviewCount</code> for read-heavy workloads. | ✓ Use when | For similar entities like <code>cardPayment</code> and <code>upiPayment</code> sharing base fields. |
| ➤ Example | <code>orderTotal</code> , <code>reviewCount</code> stored and refreshed. | ➤ Example | Single payments collection with type field. |
| ⚠ Watch out | Refrain computed fields consistently. | ⚠ Watch out | Document type clearly. |

COMMON ANTI-PATTERNS

-  **UNBOUNDED ARRAYS**
Arrays that grow without limit will hurt performance.
-  **TOO MANY JOINS/LOOKUPS**
Overusing lookups adds latency and complexity.
-  **HUGE DOCUMENTS**
Very large documents exceed limits and slow reads/writes.
-  **INDEXING EVERYTHING**
Too many indexes slow down writes.
-  **MODELING LIKE TABLES**
Design for access patterns, not relational schemas.

1 INDEX TYPES

TYPE	USE CASE	NOTE
 Single field	Filter or sort on one field.	Use for high-cardinality fields.
 Compound	Filter, sort, or range on multiple fields.	Order matters. Put most selective fields first.
 Multikey	Index array fields.	One multikey field per compound index.
 Text	Keyword text search across fields.	One text index per collection.
 TTL	Auto-expire documents after a time.	TTL index on a date field only.
 Partial	Index a subset of documents.	Reduce index size and improve performance.

2 INDEX RULES

- ✓ Index the fields you filter and sort on.
- ✓ For compound indexes think: equality → sort → range.
- ✓ Too many indexes slow down writes.
- ✓ Review explain() plans for slow queries.
- ✓ Auto-index creation is useful in dev but review carefully for prod.

3 ANNOTATION EXAMPLES

```
// Unique index on email
@Indexed(unique = true)
private String email;

// Compound index
@CompoundIndex(name = "cust_status_created_idx",
    def = "{ 'customerId': 1, 'status': 1, 'createdAt': -1 }")

// Text index
@TextIndexed
private String description;

// TTL index (expire after 7 days)
@Indexed(expireAfter = "7d")
private Instant expiresAt; // TTL index on date field
```

4 QUERY TUNING

- ✓ Use projections to return only the fields you need.
- ✓ Page with Sort + Pageable or cursor-based approach for large feeds.
- ✓ Avoid skip/limit for very deep pages when possible.
- ✓ Use aggregation pipelines for reporting and analytics.
- ✓ Keep \$lookup limited and intentional.

5 SPRING DATA EXAMPLES

```
Projected repository query interface
public interface UserView {
    String getId();
    String getName();
    String getEmail();
}

Pageable repository method
Page<Order> findByCustomerIdAndStatus(
    String customerId, String status, Pageable pageable);

MongoTemplate aggregation skeleton
Aggregation agg = Aggregation.newAggregation(
    match(Criteria.where("status").is("ACTIVE")),
    group("customerId").sum("total").as("total"),
    sort(Sort.by(Sort.Direction.DESC, "total"))
);
AggregationResults<ReportRow> results =
    mongoTemplate.aggregate(agg, "orders", ReportRow.class);
```

6 SCALING NOTES

- ✓ Read-heavy workloads: scale out with replica sets and read preferences.
- ✓ Write-heavy workloads: optimize writes and keep indexes lean.
- ✓ Shard key must match your access pattern.
- ✓ Choose a high-cardinality, stable shard key.
- ✓ Avoid hotspot shard keys (e.g., sequential timestamps).
- ✓ Keep document size and array growth under control.

7 PERFORMANCE QUICK WINS



Measure first.
Don't guess.



Index only what
you need.



Project less
data.



Aggregate
smartly.



Monitor slow
queries.



SAMPLE application.yml

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://user:***@cluster0.example.mongodb.net/shop?retryWrites=true&w=majority&appName=shop-service
      database: shop
      auto-index-creation: false
      uuid-representation: standard
```



SPRING BOOT PROPERTIES

Property	What it does	Typical production note
spring.data.mongodb.uri	Full connection string, usually preferred over host/port.	Use for Atlas and advanced options (pooling, timeouts, TLS, read preference, etc.).
spring.data.mongodb.database	Default database name.	Set the application's default database.
spring.data.mongodb.authentication-database	Auth database when different.	Use when credentials live in a separate database (e.g., admin).
spring.data.mongodb.auto-index-creation	Whether annotated indexes are auto-created.	Usually false in production; create and review indexes deliberately.
spring.data.mongodb.uuid-representation	How UUIDs are encoded (BSON representation).	Use standard and keep consistent across all services.
spring.data.mongodb.field-naming-strategy	Custom field naming strategy.	Use only if you need a specific naming convention (e.g., snake_case).
spring.data.mongodb.gridfs.bucket	GridFS bucket name when storing files.	Defaults to "fs". Override only if you need separate buckets.
spring.data.mongodb.repositories.type	Repository mode (e.g., auto).	Leave as auto for standard MongoDB repositories.



COMMON URI OPTIONS

Option	What it means	Production note
retryWrites=true	Retry supported write operations on network errors.	Keep enabled for improved write reliability (idempotent writes only).
w=majority	Write concern: wait for majority acknowledgment.	Recommended default for durability and consistency.
appName=shop-service	Application name shown in MongoDB logs and monitoring.	Set a meaningful name for observability and troubleshooting.
maxPoolSize	Maximum number of concurrent connections in the pool.	Size based on load tests and instance capacity.
minPoolSize	Minimum number of connections kept in the pool.	Set > 0 to reduce connection warm-up latency.
connectTimeoutMS	Max time to establish a connection.	Keep reasonable (e.g., 5,000–10,000 ms).
socketTimeoutMS	Max idle time for a socket (read/write).	Prevent hanging operations (e.g., 30,000–60,000 ms).
serverSelectionTimeoutMS	Max time to select a server from the cluster.	Typically 5,000–10,000 ms.
readPreference=primary or primaryPreferred	Route reads to primary or prefer primary in replica sets.	Use primary for strong consistency; primaryPreferred for availability.
readConcernLevel	Consistency level for reads (e.g., local, majority).	Use majority for stronger guarantees when needed.
tls=true	Enable TLS/SSL encryption.	Always enable in production.



PROPERTY RULES



1. Prefer uri property for flexibility and advanced options.



2. Externalize secrets (env/secret manager). Never commit passwords.



3. Tune pool and timeouts based on load tests and real traffic.



4. Keep write concern intentional; balance durability and latency.



5. Document all non-default settings for consistency across environments.

1 CONSISTENCY & TRANSACTIONS

- ✓ Single-document writes are atomic.
- ✓ Use multi-document transactions only when needed.
- ✓ Spring Data MongoDB supports transactions with TransactionTemplate / TransactionalOperator.
- ✓ Transactions require the right cluster setup.
- ✓ Design to keep transaction scope small.

2 WRITE / READ SETTINGS

-  **Write Concern:** Controls acknowledgment level. Use majority for durability.
-  **Read Concern:** Defines the level of isolation. Use majority for consistent reads.
-  **Read Preference:** Choose primary, secondary, nearest, or tag sets to balance load.
-  **Primary Reads Matter When:** You need the latest data (e.g., after writes) or strict consistency.

3 SECURITY

- ✓ Use SCRAM or platform auth.
- ✓ Enable TLS.
- ✓ Use least-privilege users.
- ✓ Never hardcode secrets.
- ✓ Restrict network access.
- ✓ Rotate credentials.

4 OBSERVABILITY

- ✓ Monitor connection pool, command latency, slow queries, index usage, replication lag, disk and memory.
- ✓ Use logs, metrics, tracing, and MongoDB/Atlas monitoring.
- ✓ Review explain plans.

5 BACKUP & OPERATIONS

- ✓ Run regular backup and restore drills.
- ✓ Define TTL and archival strategy.
- ✓ Review indexes periodically.
- ✓ Review schema and data growth.
- ✓ Perform rolling upgrades.
- ✓ Plan capacity and set alerts.

SPRING BOOT PRODUCTION CHECKLIST

- ☐ 1. Connection URI externalized (no secrets in code).
- ☐ 2. Indexes reviewed and in place.
- ☐ 3. Auto-index creation policy decided.
- ☐ 4. Projections used to limit data transfer.
- ☐ 5. Transactions used minimally and scoped.
- ☐ 6. Connection pool and timeouts tested.
- ☐ 7. Monitoring, metrics, and alerts enabled.
- ☐ 8. Backup and restore tested.
- ☐ 9. Security hardened (auth, TLS, network).
- ☐ 10. Realistic load test completed.

REMEMBER



Model for access patterns.



Keep documents bounded.



Embed intentionally.



Index deliberately.



Measure before scaling.